

SE373 · BUỔI 03 · KHOA CÔNG NGHỆ PHẦN MỀM · UIT

Agent fundamentals

Kỹ thuật xây dựng hệ thống Agentic AI

GIẢNG VIÊN LÝ THUYẾT

TS. Đỗ Trọng Hợp · ThS. Ngô Ngọc Đăng Khoa · ThS. Phạm Hoàng Hải

GIẢNG VIÊN THỰC HÀNH

Bùi Cao Doanh · Dương Nguyễn Phương Nam · Nguyễn Hiếu Nghĩa

Nguyễn Ngọc Quý · Nguyễn Thị Hoàng Anh · Quan Chí Khánh An

NỘI DUNG

Năm phần của buổi học

Kết thúc buổi: dựng được vòng lặp agent có điều kiện dừng kiểm chứng được, và đọc trace để chỉ ra chỗ hỏng.

- 01 | **Vòng lặp agent**
Agent là gì, bốn thành phần, ranh giới model và harness.
- 02 | **ReAct và các mẫu suy luận**
ReAct, plan-then-execute, reflection, và cách chọn mẫu.
- 03 | **Điều kiện dừng**
Năm kiểu dừng, tiêu chí hoàn thành, phát hiện lặp.
- 04 | **Agent debugging**
Bốn vấn đề thường gặp, dấu hiệu trong log, sửa lỗi với harness.
- 05 | **Bài tập**
Xác định lỗi và cài đặt Harness

Ba phần đầu dựng vòng lặp, **phần 04 là phần khó nhất**: đọc trace để chỉ ra chỗ hỏng.

PHẦN 01 / 05

Vòng lặp agent

Bài 03 · Agent fundamentals

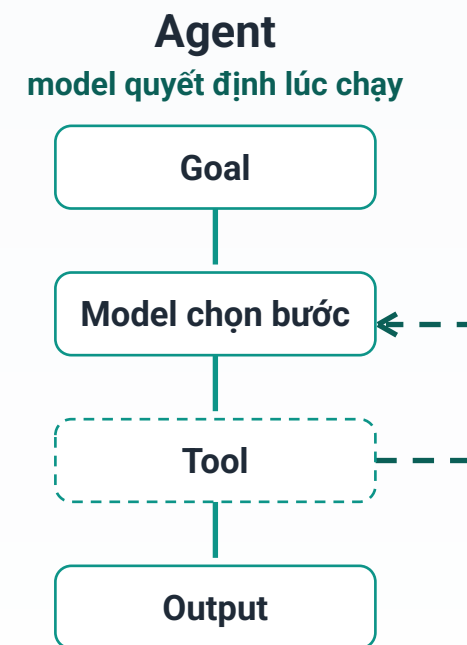
ĐỊNH NGHĨA

Agent

Phần mềm có khả năng tự chủ hoạt động, đưa ra quyết định để đạt mục tiêu mà không cần con người can thiệp liên tục.

- **Tự chủ:** Tự quản lý và quyết định bước đi tiếp theo.
- **Chủ động:** Tự lên kế hoạch và khởi xướng hành động.
- **Phản ứng:** Nhận thức môi trường và thích ứng kịp thời.

Ví dụ: “Tìm vé máy bay → chuyển đầu hết chỗ → agent xem kết quả và chọn tìm chuyến khác.”



CẤU TẠO

Bốn thành phần tối thiểu

01

Goal

Mục tiêu cần đạt.

02

Tools

Các công cụ thêm cho
Agent

03

Loop

Đưa kết quả hành động trở
lại làm đầu vào.

04

Termination

Cơ chế quyết định dừng hay
chạy tiếp.

`agent = goal + tools + loop + termination`

Thiếu **termination** là tổn kém nhất.

ĐỊNH NGHĨA

Goal

Mô tả trạng thái cần đạt, không phải danh sách bước cần làm.

KHÔNG PHẢI GOAL

Tóm tắt đoạn văn này

LÀ GOAL

Tìm commit làm hỏng test_checkout và mở issue

Một lệnh một bước không cần **vòng lặp**.

HAI THÀNH PHẦN

Tools và Loop

TOOLS

Hàm bên ngoài mà model gọi được qua giao thức tool calling, có tên, mô tả và schema tham số.

- Không có tool thì model chỉ sinh văn bản.

LOOP

Chu trình đưa kết quả của hành động vừa thực hiện trở lại làm đầu vào cho lần suy luận kế tiếp.

- Một lần gọi tool chưa phải agent.

Gọi tool → Sinh thêm dữ liệu → Quan sát/Đánh giá → **Lặp/Dừng.**

ĐỊNH NGHĨA

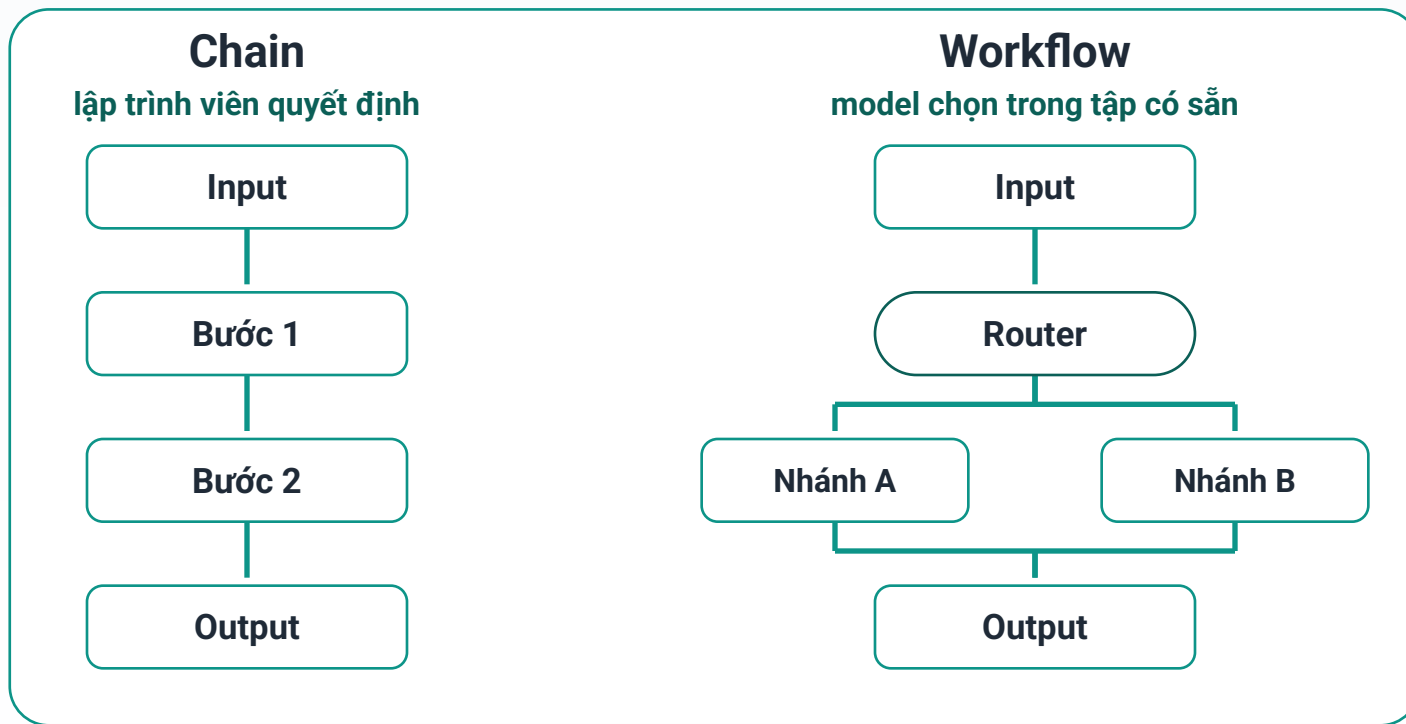
Termination

Cơ chế quyết định **chạy tiếp hay dừng lại**.

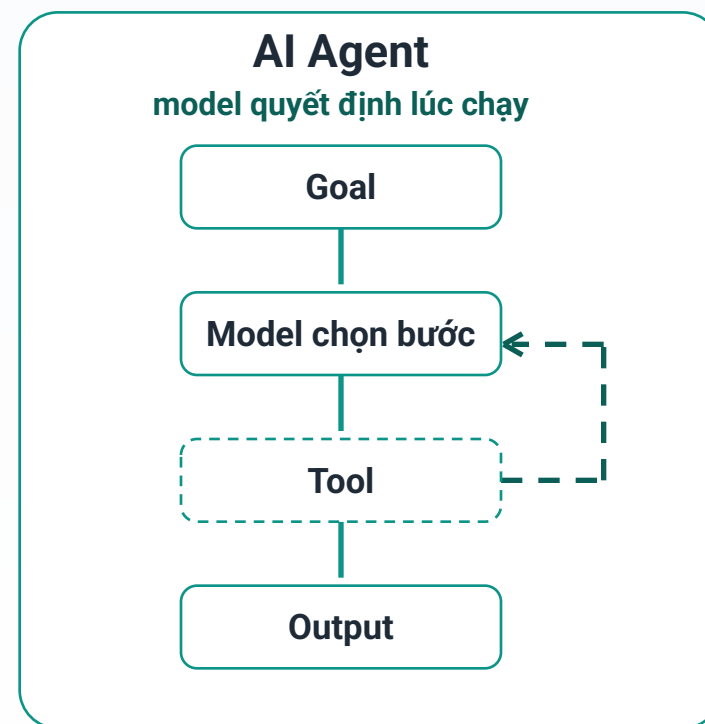
- Gồm điều kiện hoàn thành và các giới hạn cứng.
- Thiếu nó, agent không có điểm kết thúc và chi phí không có trần.

PHÂN LOẠI

Agent truyền thống vs AI Agent



Agent truyền thống

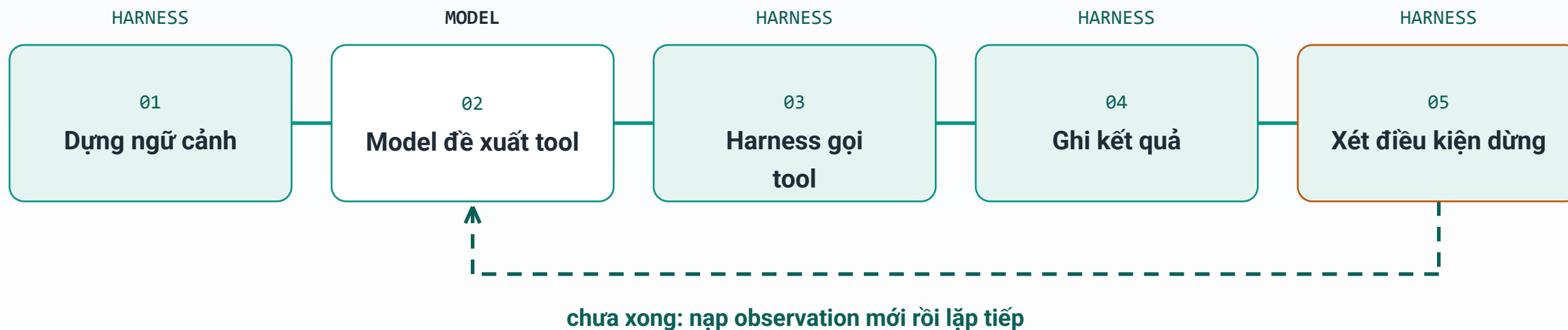


AI Agent

Câu hỏi phân loại: **bước tiếp theo do ai quyết định.**

SƠ ĐỒ TRỰC

Vòng lặp AI Agent

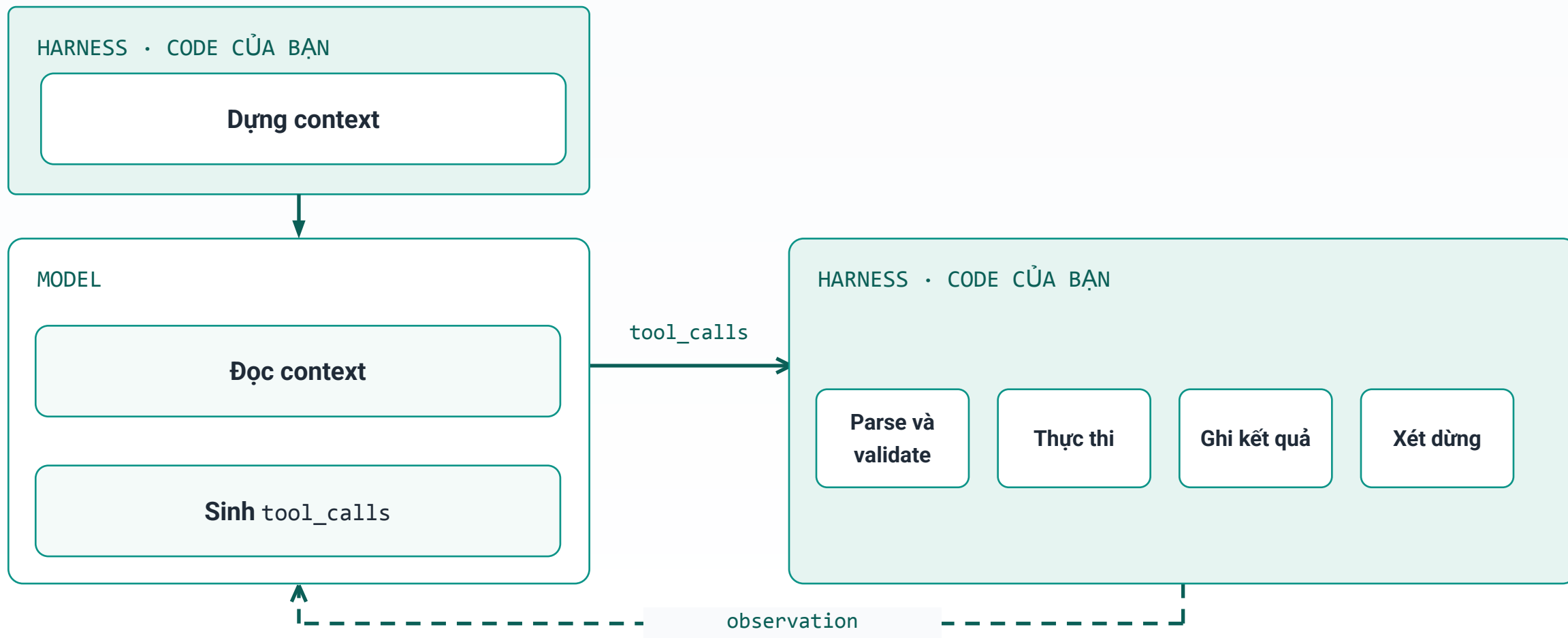


Ở bước **"Xét điều kiện dừng"** có 3 hành động: lặp tiếp, chờ người, hoặc thoát.

Chỉ bước 02 là của model. Bốn bước còn lại là code do bạn viết.

RANH GIỚI

Ranh giới model và harness



STATE

Tool call là dữ liệu

```
{  
  "name": "weather_forecast",  
  "args": {"town": "Quy Nhơn"},  
  "id": "call_w1"  
}
```

```
AIMessage  
  tool_calls=[  
    {name: weather_forecast,  
     args: {"town": "Quy Nhơn"},  
     id: call_w1}]
```

```
ToolMessage  
  tool_call_id=call_w1  
  content={"weather": "nắng"}
```

Sử dụng **id** để đưa kết quả lịch sử hội thoại.

OBSERVATION

Chuẩn hóa structured-output

Output observation kém

```
<div id="mw-content">  
<p>Nha Trang, thuộc  
<b><a rel="nofollow"  
href="/wiki/Khanh_Hoa">  
Khánh Hoà</a></b>, nổi  
tiếng với bãi biển dài
```

Output observation tốt

```
{"town": "Nha Trang",  
  "weather": "nắng",  
  "temperature": 31}
```

Output của LLM nên là dạng structured-output theo JSON.

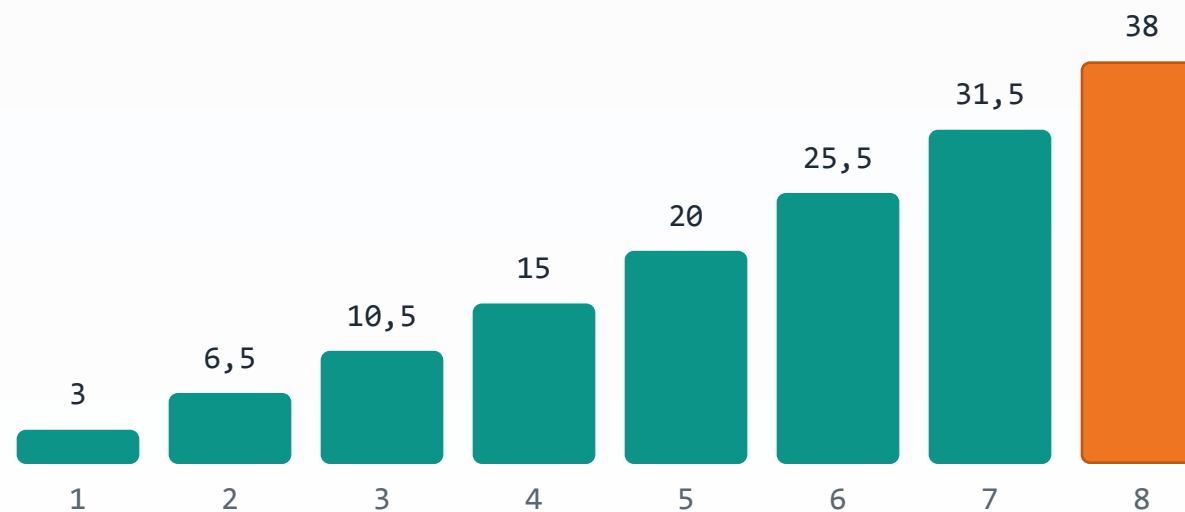
CHI PHÍ

Chi phí của lịch sử

Mỗi vòng nạp lại toàn bộ lịch sử, mà lịch sử dài thêm sau mỗi vòng.

- Tổng chi phí tăng theo bình phương số vòng.
- Gấp đôi số vòng thì chi phí gấp khoảng bốn lần.

Token gia tăng theo số vòng lặp



Số giả định để tạo trực giác: 3.000 token nền, mỗi vòng thêm 500 token lịch sử.

GIỚI HẠN CỨNG

Ngân sách vòng lặp

01

Bước

Tối đa bao nhiêu vòng.

02

Token

Tối đa bao nhiêu token cho cả phiên.

03

Thời gian

Tối đa bao nhiêu giây.

04

Chi phí

Trần chi phí cho một tác vụ.

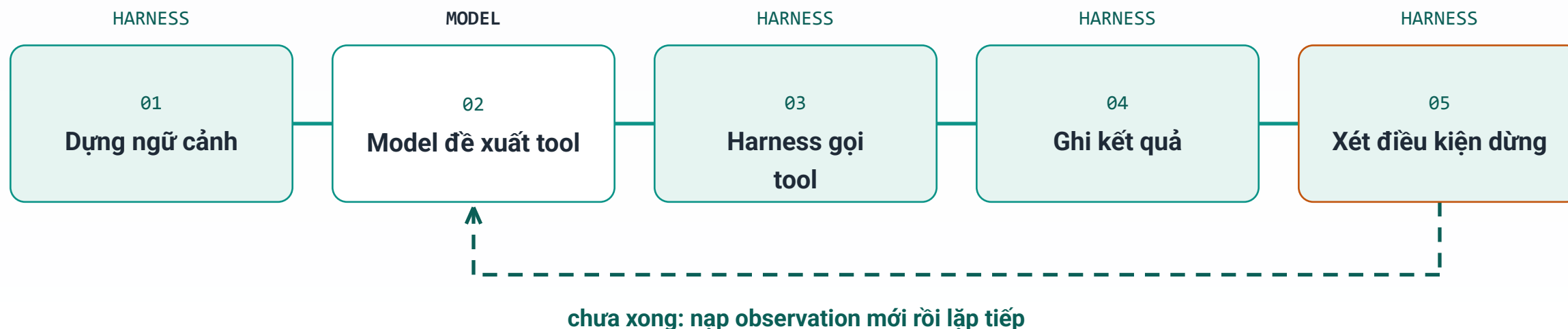
CHỐT PHẦN 1

Tổng kết vòng lặp Agent

- 01 Agent là model chọn bước tiếp theo lúc chạy, trong giới hạn của harness.
- 02 `agent = goal + tools + loop + termination.`
- 03 Model sinh lời gọi tool; harness thực thi, ghi state và xét điều kiện dừng.
- 04 Thiết kế cách observation (output, điều kiện dừng...)

ReAct và các mẫu suy luận

Cách **tổ chức suy luận** bên trong vòng lặp.

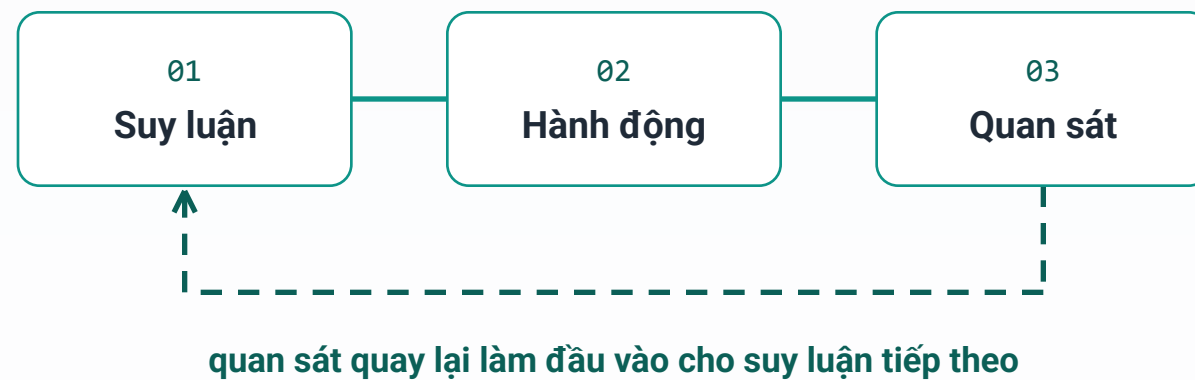


MẪU SUY LUẬN

ReAct

ReAct là kiến trúc cốt lõi giúp AI Agent liên tục **suy luận, hành động và quan sát** thực tế.

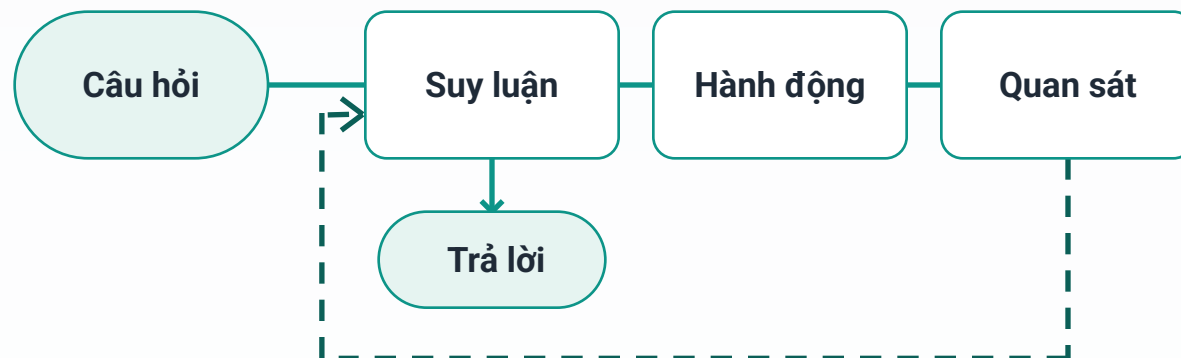
- Reasoning + Acting, Yao và cộng sự, 2022.
- Không phải thư viện, cũng không phải một model.



PHÂN LOẠI

Ba cách tổ chức suy luận

- **Chain of thought**: suy luận nhiều bước nhưng không sử dụng môi trường ngoài.
- **Action only**: hành động nhưng không có bước suy luận tường minh.
- **ReAct**: suy luận, hành động, quan sát, rồi suy luận tiếp.



LÝ DO

Vì sao ReAct hiệu quả hơn

- Dữ kiện từ môi trường đi vào giữa chuỗi suy luận, thay vì chỉ có trí nhớ của model.
- Hướng đi được điều chỉnh sau mỗi quan sát, không cố định từ đầu.
- **Trace ghi lại từng vòng**, nên chẩn đoán được khi sai.

VÍ DỤ

Vòng lặp đặt vé máy bay



vòng 3

NGŨ CẢNH ĐÃ CÓ

12 chuyến phù hợp

trống thứ ba → thứ năm

đã xác nhận · 4XJ2

KÍCH THƯỚC NGŨ CẢNH

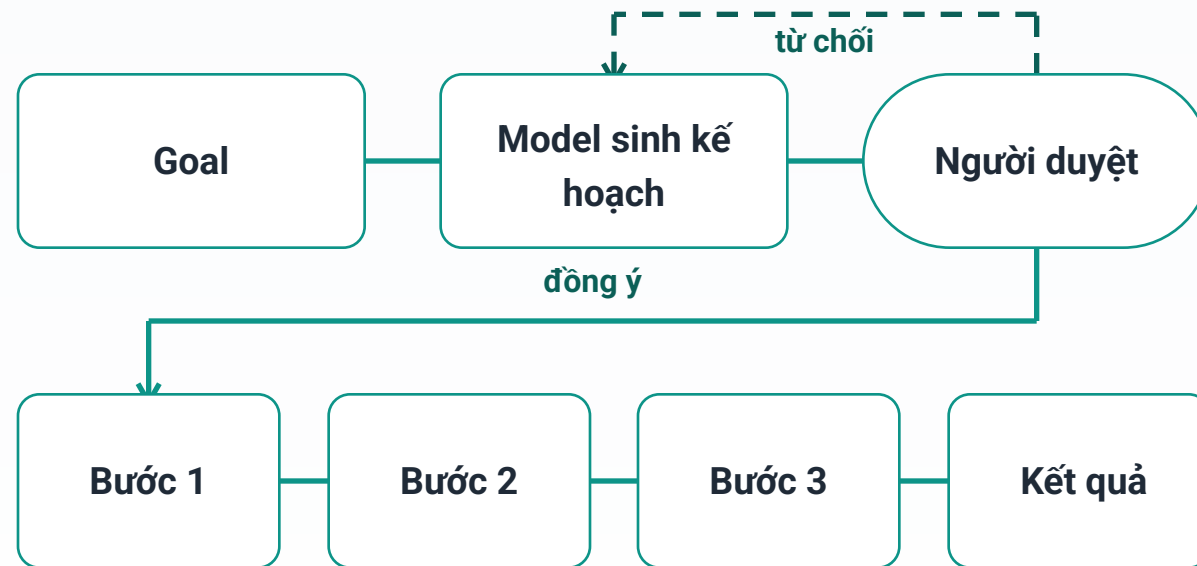
4.100 token

Agent không trả lời trong một lượt: nó quay vòng cho tới khi xong, và mỗi vòng gửi lại toàn bộ lịch sử.

MẪU KẾ HOẠCH

Plan-then-execute

Plan-then-execute là mẫu **gọi model một lần để sinh trọn kế hoạch, rồi thực thi từng bước** theo kế hoạch đó.



Ưu thế quyết định là kế hoạch nhìn thấy được trước khi chạy, nên **duyệt được và ước lượng chi phí được**.

ĐÁNH ĐỐI

Được và mất

PROS

- Giữ vững định hướng khi xử lý tác vụ dài.
- Tiết kiệm chi phí và hỗ trợ chạy song song.
- • Dễ dàng dùng mô hình nhỏ để thực thi.

CONS

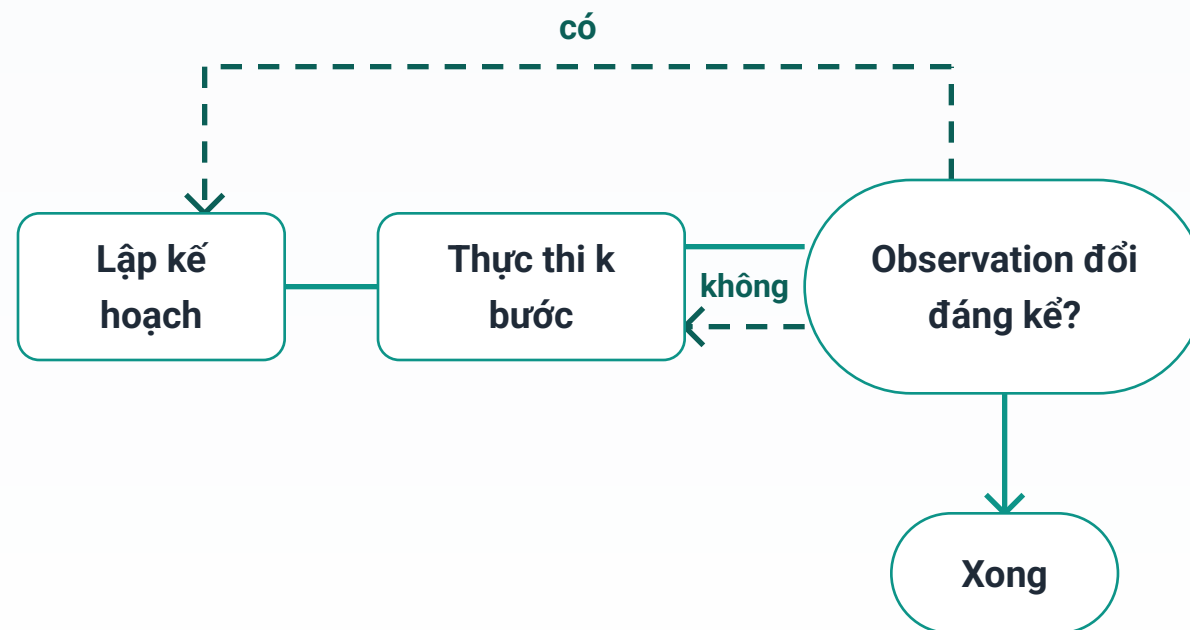
- Thiếu linh hoạt khi tình huống thay đổi bất ngờ.
- Lỗi ở bước đầu sẽ làm hỏng toàn bộ sau.
- Tốn nhiều thời gian ban đầu để lập kế hoạch.

Đây là đánh đổi giữa **khả năng duyệt trước** và **khả năng thích nghi**.

SƠ ĐỒ TRỰC

Mẫu lai “ReAct + Plan”

Lập kế hoạch, **thực thi vài bước**, rồi **lập lại kế hoạch** dựa trên những gì vừa quan sát.



LangChain 1.x hỗ trợ mẫu này qua `TodoListMiddleware`.

MẪU TỰ PHẢN TỈNH

Reflection

Reflection là mẫu kiến trúc cho phép AI Agent tự đánh giá và sửa lỗi kết quả của chính mình.



- Tự động phát hiện và sửa lỗi sai.
- Nâng cao đáng kể chất lượng đầu ra.

TỔNG HỢP

Bảng chọn mẫu

MẪU	CHỌN KHI	RỦI RO CHÍNH
ReAct	Không đoán được số bước	Lặp vô hạn, trôi mục tiêu
Plan-then-execute	Cần duyệt trước	Kế hoạch lỗi thời
Lai	Tác vụ dài, môi trường biến động	Khó debug hơn
Reflection	Có tín hiệu kiểm chứng ngoài	Nhân đôi chi phí

Bảng này dùng khi chọn kiến trúc cho đề án.

QUIZ

Làm quiz ngay tại lớp



Quét mã, hoặc mở forms.gle → **SE373 buổi 03**

10 câu · 5 phút · làm xong ta chữa ngay tại lớp

DEMO

DEMO	CHẠY LỆNH	PHẢI THẤY ĐƯỢC
Demo 1	demo1_create_agent.py	Vòng lặp agent, dừng khi thôi gọi tool
Demo 2	demo2_dieu_kien_dung.py	Cài đặt 3 điều kiện dừng
Demo 3	demo3_doc_trace.py A	Sửa lỗi dựa vào trace log

DEMO 01

Khởi tạo Agent với Langchain · `create_agent`

- LangChain và LangGraph 1.0 phát hành 22/10/2025.
- API chuẩn hiện nay: `create_agent` trong `langchain.agents`.

```
agent = create_agent(  
    model=os.environ["SE373_MODEL"],  
    tools=TOOLS,  
    system_prompt="Chỉ dùng tool để lấy thông tin, kể cả tên điểm đến.",  
    middleware=[ModelCallLimitMiddleware(run_limit=8, exit_behavior="end")],  
)  
result = agent.invoke({"messages": [{"role": "user", "content": "CAU_HOI}]})
```

SƠ SÁNH

Có tool chưa chắc dùng tool

Không có system

prompt
Agent gọi thẳng:

```
weather_forecast("Nha Trang")  
weather_forecast("Đà Nẵng")
```

Tên điểm đến lấy từ trí nhớ của model.

Có system

prompt
Agent gọi:

```
search_travel_info(...)  
→ weather_forecast(tên từ kết quả tìm kiếm)
```

Có tool **không có nghĩa là** agent sẽ dùng tool.

TỔNG KẾT

Tổng kết phần 2

- 01 Mẫu ReAct: Suy luận → Hành động → Quan sát kết quả → Suy luận tiếp.
- 02 Mẫu Plan-then-execute: Lập bản kế hoạch tổng thể gồm nhiều bước → Thực thi từng bước.
- 03 Mẫu Reflection: Tự tạo ra kết quả → Tự đánh giá tìm lỗi → Sửa lại.
- 04 Có tool chưa chắc agent dùng tool.

PHẦN 03 / 05

Điều kiện dừng

Bài 03 · Agent fundamentals

ĐỊNH NGHĨA

Điều kiện dừng (**Termination Condition**)

Termination Condition là tiêu chí để Agent kết thúc vòng lặp thực thi khi đạt mục tiêu hoặc chạm giới hạn.

- Mặc định của framework: dừng khi model thôi gọi tool.

SƠ ĐỒ QUYẾT ĐỊNH

5 điều kiện dừng



TRỤC CỦA PHẦN NÀY

Checklist harness chạy sau mỗi vòng

#	CHẠY KHI NÀO	KIỂM GÌ	KẾT THÚC KIỂU	LOẠI
0	trước khi thực thi tool	quyền hạn của hành động sắp làm	Cần con người	bình thường
1	sau khi có observation	tiêu chí hoàn thành	Đạt mục tiêu	bình thường
2	sau khi có observation	(tool, args) có trùng vòng trước	Phát hiện lặp	bất thường
3	sau khi có observation	đại lượng tiến triển có nhúc nhích	Bế tắc	bất thường
4	sau khi có observation	vòng · token · thời gian · tiền	Hết ngân sách	bất thường

Ngân sách kiểm cuối cùng: đặt nó lên đầu thì mọi lỗi đều báo về là hết ngân sách, và bạn mất chẩn đoán.

SƠ SÁNH

Các cách cài đặt điều kiện dừng

Sensor computational

- Test, linter, type checker.
- Xác định, chạy trong mili giây.
- Không tốn token.

Sensor

inferential

- Một model chấm output của model.
- Không xác định, tốn thêm chi phí.
- Dừng khi không có cách đo máy móc.

Ưu tiên **sensor computational** bất cứ khi nào có thể.

ĐIỀU KIỆN DỪNG 01 / 05

Điều kiện dừng 1 · Đạt mục tiêu

Model thôi gọi tool, nghĩa là nó tự cho rằng đã xong.

```
V1  search_flights("SGN","DAD","07/10")    → 12 chuyến
V2  check_seat("VN122")                     → 3 ghế · 1.850.000
V3  book_seat("VN122","12A")                 → 4XJ2 · held
V4  pay("4XJ2","corp_card")                  → paid
V5  get_booking("4XJ2")                      → confirmed · paid · 1.850.000
```

- Đây là kiểu dừng duy nhất ta mong muốn.
- Phải kiểm chứng kết quả trước khi tin.

Cũng là kiểu **nguy hiểm nhất khi sai**, vì nó trông giống thành công.

ĐIỀU KIỆN DỪNG 02 / 05

Điều kiện dừng 2 · Hết ngân sách

Harness đếm và thấy đã chạm trần bước, token, thời gian hoặc tiền.

```
V2  check_seat("VN122")    → 2.310.000  X
V3  check_seat("VJ604")    → 2.080.000  X
V4  check_seat("QH118")    → 2.450.000  X
...  mỗi vòng một chuyến khác · không lặp · vẫn có tiến triển
V12 check_seat("VN134")    → 2.190.000  X   chạm trần 12 vòng
```

- Xác định, không phụ thuộc model.
- LangGraph: `GraphRecursionError`. LangChain: `ModelCallLimitMiddleware`.

Dừng bất thường: phải log và trả kết quả dở dang cho người.

ĐIỀU KIỆN DỪNG 03 / 05

Điều kiện dừng 3 · Phát hiện Lặp

Agent gọi lại cùng một hành động hoặc nhận lại cùng một kết quả mà không tiến gần hơn tới mục tiêu.

```
V2  check_seat("VN122")  → {"error": "timeout"}
V3  check_seat("VN122")  → {"error": "timeout"}
V4  check_seat("VN122")  → {"error": "timeout"}
(tool, args) trùng ba lần · ngân sách vẫn còn 8 vòng
```

- Framework không có sẵn cơ chế này.
- Đây là **code bạn phải tự viết**.

Demo 2 · vòng V4 là lúc bộ phát hiện lặp phải báo động.

So (tool, args), không so observation. Gọi lại get_booking để chờ confirmed là polling hợp lệ.

ĐIỀU KIỆN DỪNG 04 / 05

Điều kiện dừng 4 · Bế tắc

Agent **không lặp nhưng cũng không tiến**: mỗi vòng thử một tool khác và đều thất bại.

```
V4  search_flights(..., "07/10")  chiều thứ Ba    → sai giờ
V5  search_flights(..., "08/10")  sáng thứ Tư   → sai ngày
V6  check_seat("VJ612")           hãng khác     → sai giá
V7  search_flights("SGN","HUI")   bay Huế      → sai điểm đến
```

ràng buộc đã thoả: 1/2 · đứng yên suốt 6 vòng

- Khó thấy hơn lặp, vì trace nhìn rất bận rộn.
- Phát hiện bằng cách đo một đại lượng tiến triển của bài toán.

Mỗi vòng thử một tool khác và đều thất bại.

ĐIỀU KIỆN DỪNG 05 / 05

Điều kiện dừng 5 · Cần thông tin/approval

Agent chạm tới một hành động vượt thẩm quyền và dừng lại chờ phê duyệt.

```
V3 model định gọi: book_seat("VN122","12A") · 1.950.000đ · không hoàn
Đang ở đâu    : 12 chuyến, rẻ nhất thoả giờ là VN122
Định làm gì   : đặt ghế 12A · 1.950.000đ · vé không hoàn
Vì sao hỏi    : vượt hạn mức 1.500.000đ và không hoàn được
```

- Ví dụ: xoá dữ liệu, chuyển tiền, gửi mail cho khách, merge vào `main`.

Kiểm quyền chạy trước khi thực thi, bốn kiểm kia chạy sau khi có observation.

PHÂN LOẠI DỪNG

Bình thường và bất thường

Kiểu DỪNG	AI PHÁT HIỆN	BÌNH THƯỜNG	
Đạt mục tiêu	Model	Có	Kết A · vé confirmed, đã trả tiền
Hết ngân sách	Harness đếm	Không	Kết B · dò 12 chuyến, đều trên 2 triệu
Phát hiện lặp	Harness so sánh	Không	Kết C · check_seat lỗi ba lần liên
Bế tắc	Harness đo tiến triển	Không	Kết D · đổi hướng 6 vòng, vẫn 1/2 ràng buộc
Cần con người	Harness kiểm quyền	Có	Kết E · vé không hoàn, vượt hạn mức

Dừng bất thường mà im lặng thì biến một lỗi thấy được thành một lỗi ẩn.

ĐỊNH NGHĨA

Tiêu chí hoàn thành (Điều kiện dừng)

Tiêu chí hoàn thành là các **quy tắc lập trình khách quan** được hệ thống bên ngoài kiểm chứng để xác nhận tác vụ đã xong, hoàn toàn độc lập với phán đoán của model.

```
# tiêu chí hoàn thành của bài đặt vé
get_booking(code).status == "confirmed" and paid == True
and price <= 2_000_000
and depart_date == "2026-10-07" and depart_time < "12:00"
```

Phân biệt cách đánh giá "Đã xong":

- ❌ **Sai (Chủ quan):** Phụ thuộc vào model tự tuyên bố (Ví dụ: *"Model cảm thấy đã thu thập đủ thông tin"* thì dừng).
- ✅ **Đúng (Khách quan):** Dùng **code để kiểm tra** chính xác trạng thái.

PHÂN LOẠI

Bốn dạng tiêu chí kiểm chứng được

01

Vị từ chạy bằng code

```
pytest exit code 0, HTTP  
200
```

```
get_booking(c).status ==  
"confirmed"
```

02

Schema hợp lệ

Output parse được theo
Pydantic.

*Kết quả book_seat parse
được theo model Booking.*

03

Kiểm chứng chéo

Agent báo đã tạo issue thì
gọi API đọc lại.

*Giá trong get_booking khớp
giá đã thấy ở check_seat.*

04

Người duyệt

Khi ba cách trên không áp
dụng được.

*Vé không hoàn phải có
duyet trước khi pay.*

Dạng một xác định, chạy trong mili giây, không tốn token: đây là sensor computational.

PHÁT HIỆN LẶP

Ba tín hiệu phát hiện lặp

- **Trùng action:** cùng (tool, args) xuất hiện lại trong cửa sổ vài vòng.
- **Trùng observation:** tham số khác nhau nhưng kết quả giống hệt.
- **Không tiến triển:** một đại lượng của bài toán đứng yên qua N vòng.

Tín hiệu ba bắt được kiểu lặp mà hai tín hiệu đầu bỏ sót: agent đổi tool mỗi vòng nhưng vẫn đứng yên.

CODE

Bộ phát hiện lặp

```
class LoopDetector:
    def __init__(self, window=6, repeat_k=2, stall_n=5):
        self.recent = deque(maxlen=window)      # chỉ số cửa sổ gần
        self.k, self.n, self.last, self.stall = repeat_k, stall_n, None, 0

    def check(self, tool, args, progress):
        fp = (tool, repr(sorted(args.items())))
        if self.recent.count(fp) + 1 >= self.k: return "LOOP"
        self.recent.append(fp)
        self.stall = self.stall + 1 if progress == self.last else 0
        self.last = progress
        return "STALL" if self.stall >= self.n else None
```

Tham số progress do bạn truyền vào, là đại lượng của bài toán, ví dụ số test còn fail. Không có bộ phát hiện lặp tổng quát. Code: demo/lib/harness.py.

DEMO

DEMO	CHẠY LỆNH	PHẢI THẤY ĐƯỢC
Demo 1	demo1_create_agent.py	Vòng lặp agent, dừng khi thôi gọi tool
Demo 2	demo2_dieu_kien_dung.py	Cài đặt 3 điều kiện dừng
Demo 3	demo3_doc_trace.py A	Sửa lỗi dựa vào trace log

BÀN GIAO

Bàn giao cho con người

- **Trạng thái:** đã làm tới đâu, hành động nào đã có tác dụng phụ.
- **Những gì đã thử:** hướng nào đã hỏng và vì sao.
- **Câu hỏi cụ thể:** "Dùng bảng orders hay orders_v2?"

"Bàn giao tốt là bàn giao mà người nhận trả lời được trong 30 giây."

Đây là lớp harness handoff, lớp thứ 12.

CHỐT PHẦN 3

Tổng kết phần 3

- 01 Năm điều kiện dừng (termination condition)
- 02 Tiêu chí hoàn thành phải kiểm chứng được từ logic code.
- 03 Phát hiện lặp là code bạn tự viết, framework không làm hộ.

Agent Debugging

Bài 03 · Agent fundamentals

QUY TRÌNH

4 BƯỚC DEBUG AGENT

Quy trình áp dụng chung cho mọi bài toán. (Failure mode: Kiểu lỗi mang tính lặp lại của agent)

BƯỚC 01

Tái hiện

- Chạy lại với đúng đầu vào cũ.
- Lưu toàn bộ nhật ký (log) từng bước.

BƯỚC 02

Khoanh vùng

- Tìm vòng lặp sai đầu tiên.
- Tuyệt đối không nhìn kết quả cuối.

BƯỚC 03

Xác định loại lỗi

- Đối chiếu 4 dấu hiệu đặc trưng.
- Phân loại chính xác kiểu lỗi.

BƯỚC 04

Đặt kiểm tra

- Thêm code chặn tại vòng bị lỗi.
- Chạy lại để xác nhận giả thuyết.

Kiểu lỗi lặp lại được của agent thường gọi là failure mode.

BƯỚC 03 · BẢNG TRA

Các lỗi thường gặp

Dấu hiệu trong Log

Gọi một công cụ lặp lại, kết quả y hệt.

TÊN LOẠI LỖI

Lặp không tiến bộ

ĐẶT HARNESS

Đếm số lần gọi công cụ vô ích.

Xuất hiện số liệu, ngày, ID không có trong dữ liệu thật.

Bịa đặt thông tin (Hallucination)

Đối chiếu lại kết quả trước khi gửi.

Hành động cuối bị lạc đề so với yêu cầu gốc.

Quên yêu cầu

Kiểm tra lại ràng buộc trước khi chốt.

Log trơn tru, không lặp nhưng sai từ kết luận đầu.

Tin dữ liệu sai

Ép công cụ trả về trạng thái rõ ràng.

VẤN ĐỀ 01 · DẤU HIỆU

Lặp lại nhưng không tiến bộ

Agent thử cách khác nhưng nhận lại cùng một kết quả, nên không có thông tin mới để đi tiếp.

Dấu hiệu

Gọi một công cụ lặp lại, kết quả y hệt.

Vì sao

Công cụ không trả về mã lỗi chi tiết

Code chặn

Đếm số lần gọi không tiến triển để ngắt luồng hoặc báo người dùng.

Ví dụ: hỏi thời tiết Vũng Tàu, đổi cách viết tên rồi gọi lại 35 lần.

VẤN ĐỀ 01 · TÌNH HUỐNG

Hỏi hết giới hạn ngân sách

```
[V1] Human  "Ngày mai giao hàng ở Vũng Tàu có mưa không?"
[V1] AI      weather_forecast(town="Vung Tau")      → '{"error": "not found"}'
[V2] AI      weather_forecast(town="Vũng Tàu")      → '{"error": "not found"}'
[V3] AI      weather_forecast(town="Vung Tau City") → '{"error": "not found"}'
[V4] AI      weather_forecast(town="Vung Tau")      → '{"error": "not found"}'
...          V5 đến V35: luân phiên năm cách viết tên, kết quả y hệt
[V36] hết giới hạn 36 lượt · dừng · không có câu trả lời
```

V4 gọi lại đúng cách viết đã lỗi ở V1 · từ vòng nào đủ bằng chứng rằng agent đang lặp?

VẤN ĐỀ 01 · CÁCH XỬ LÝ

Thêm mã lỗi để agent hiểu và biết dừng

Thay "not found" bằng một lỗi nói được đường đi khác.

```
{"status": "invalid_town",  
"hint": "Gọi list_supported_towns  
để lấy danh sách hợp lệ",  
"supported_count": 63}
```

VẤN ĐỀ 02 · DẤU HIỆU

Bịa đặt thông tin (Hallucination)

Agent tự chế ra dữ liệu để trả lời

- Hỏi tổng tiền đơn hàng; công cụ chỉ trả trạng thái vận chuyển, agent vẫn trả lời giá là "1.250.000đ".
- Lỗi ngầm, chương trình vẫn **chạy trơn tru không báo lỗi**.

VẤN ĐỀ 02 · TÌNH HUỐNG

Bịa đặt thông tin (Hallucination)

```
[V1] Human  "Đơn ORD-88123 giao tới đâu rồi, và tổng tiền bao nhiêu?"
[V1] AI      get_order(order_id="ORD-88123")
[V1] Tool    {"status":"in_transit","carrier":"GHN","customer_id":"CUS-4471"}
[V2] AI      get_tracking(order_id="ORD-88123")
[V2] Tool    {"last_scan":"Kho trung chuyển Bình Dương","scanned_at":"06/09 18:40"}
[V3] AI      "Đơn đang trên đường giao, quét gần nhất tại Bình Dương lúc 18:40.
              Tổng tiền 1.250.000đ, đã gồm 30.000đ phí vận chuyển, thẻ VISA đuôi 4412."
```

Chương trình không báo lỗi · đoạn nào trong câu trả lời có vấn đề?

VẤN ĐỀ 02 · CÁCH XỬ LÝ

Đối chiếu dữ liệu với tool

Kiểm tra chéo toàn bộ số, ngày, ID, tên riêng với kết quả công cụ đã nhận

"1.250.000đ" → không có nguồn ✗

"30.000đ" → không có nguồn ✗

"VISA 4412" → không có nguồn ✗

"GHN" → call_b1 ✓

"18:40 06/09" → call_b2 ✓

VẤN ĐỀ 02 · BA CÁCH CHẶN

Agent dùng sai công cụ

AGENT LÀM GÌ	VÍ DỤ	CODE CHẶN Ở ĐÂU
Gọi công cụ không tồn tại	Gọi <code>get_order_price</code>	Chỉ cho phép gọi các tên trong danh sách
Gửi sai định dạng tham số	<code>order_id=123</code> khi cần chuỗi	Kiểm tra (validate) tham số trước khi gọi hàm.
Tự chế thêm dữ liệu (Nguy hiểm)	Chưa có dữ liệu giá vẫn nói số tiền	Đối chiếu câu trả lời với kết quả thật từ tool.

Hai lỗi đầu (1 & 2) có thể dễ dàng bắt ngay ở tầng code, nhưng lỗi thứ ba **bắt buộc phải phân tích câu trả lời** mới xác định được.

VẤN ĐỀ 03 · DẤU HIỆU

Quên yêu cầu ban đầu

Agent vẫn đang làm gì đó, nhưng kết quả không còn đáp ứng yêu cầu ban đầu.

Dấu hiệu

việc ở vòng cuối không còn liên quan tới yêu cầu lúc đầu.

Vì sao

yêu cầu lùi xa dần khi nhật ký dài ra.

Code chặn

giữ yêu cầu ở chỗ rõ ràng; trước khi trả lời, kiểm lại đúng ngày, đúng giờ, trong ngân sách.

VẤN ĐỀ 03 · TÌNH HUỐNG

Đi tìm vé, quay ra đọc đánh giá hãng

```
[V1] Human "Tìm vé TP.HCM → Đà Nẵng sáng thứ Năm, dưới 2 triệu."  
[V1] AI    search_flights("SGN","DAD","24/09") → 12 chuyến  
[V2] AI    check_seat("VN122") → 2.310.000 · quá giá  
[V3] AI    get_airline_reviews("VN") → 4.2/5  
[V4] AI    get_airline_reviews("VJ") → 3.8/5  
[V5] AI    search_flights("SGN","DAD","25/09") → chiều thứ Sáu  
[V6] AI    "Đề xuất chuyến chiều thứ Sáu, 2.400.000đ, hãng đánh giá tốt hơn."
```

Không vòng nào báo lỗi · từ vòng nào agent rời khỏi yêu cầu ban đầu?

VẤN ĐỀ 03 · CÁCH XỬ LÝ

Giữ yêu cầu ở một chỗ cố định

Ghi yêu cầu và ràng buộc thành dữ liệu, rồi kiểm lại ngay trước khi trả lời.

```
yeu_cau = {"ngay": "07/10",  
"buoi": "sang",  
"tran_gia": 2_000_000}  
check(ve, yeu_cau)    # trước khi trả lời
```

VẤN ĐỀ 04 · DẤU HIỆU

Tin vào dữ liệu sai

Công cụ trả kết quả mơ hồ → Agent tự diễn dịch sai → Mọi bước sau dù hợp lý nhưng đã sai ngay từ đầu.

Dấu hiệu	Một kết quả từ công cụ bị sai ở bước đầu, kéo theo toàn bộ suy luận phía sau nhất quán với cái sai đó
Đặc điểm	Log hệ thống "sạch" (không lặp, không báo lỗi, dừng đúng cách) nên cực kỳ khó phát hiện
Code chặn	Ràng buộc tool trả trạng thái rõ: có dữ liệu · không có dữ liệu · gọi sai · công cụ lỗi.

VẤN ĐỀ 04 · TÌNH HUỐNG

Không thấy log vì gọi sai

```
[V1] Human "Service payment trả lỗi 500 từ 14:00. Tìm nguyên nhân."
[V1] AI     search_logs(service="payment", level="ERROR") → ''
[V2] AI     list_deploys(service="payment") → dp-5512 13:47 commit 9f2a1c7
[V3] AI     get_commit(sha="9f2a1c7") → "refactor: đổi tên biến" +12 -12
[V4] AI     search_logs(service="payment", level="ERROR") → ''
[V5] AI     "Không có log ERROR nào sau 13:47 nên lỗi không đến từ ứng dụng.
            Commit chỉ đổi tên biến nên an toàn. Chuyển SRE, không cần rollback."
```

Chương trình không báo lỗi · agent đã biến '' thành khẳng định nào mà nó chưa kiểm chứng?

CÁCH XỬ LÝ

Tool phải trả kết quả rõ

Nếu tool chỉ trả về một "chuỗi rỗng", Agent sẽ không có đủ dữ kiện để biết nên làm gì tiếp theo.

```
{"status": "ok", "matched": 0, "logs": []}
```

```
{"status": "invalid_param",  
"param": "level",  
"allowed": ["error", "warn", "info"]}
```

Định dạng lại kết quả (VD: dùng JSON) để định hướng hành động cho Agent:

CHỐT PHẦN 4

Tổng kết phần 4

- 01 **Tái hiện rồi khoanh vùng:** Bắt đúng vòng sai đầu tiên, bỏ qua kết quả cuối.
- 02 **Phân loại lỗi:** Xác định chính xác 1 trong 4 kiểu lỗi.
- 03 **Cảnh giác lỗi ngầm:** Hai lỗi cuối không tự báo động (log sạch nhưng vẫn sai).
- 04 **Đặt kiểm tra:** Thêm code chặn ngay tại vòng lỗi đó và chạy lại.

PHẦN 05 / 05

Bài tập

Bài 03 · Agent fundamentals

BÀI TẬP VỀ NHÀ

BTVN#3 · Dựng agent đặt vé bằng LangChain

Tìm hiểu Langchain, LangGraph → Tạo tool mockup → Viết lớp harness cho Agent này. Nộp .py kèm báo cáo.

- 01** Đủ các lớp: ràng buộc là dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền, bàn giao;
- 02** Cài đặt Agent với 3 mẫu thiết kế: ReAct, Plan-then-Execute, Lai
- 03** Đánh giá hiệu quả của Agent với 3 mẫu thiết kế khác nhau

CHỐT BUỔI

Tổng kết buổi

- 01 Agent chọn bước tiếp theo lúc chạy, trong giới hạn của harness.
- 02 Vòng lặp có năm chặng;
- 03 ReAct, Plan-and-Execute, Reflect
- 04 5 kiểu dừng trong Agent Loop.
- 05 Agent debugging, 4 failure mode.

Buổi 4 là **harness engineering**: khung guides và sensors, và giải phẫu một agent harness đầy đủ.

BÀI 03

Q & A

Agentic AI Engineering · SE373